

Project Machine Learning

— Milestone 1 —

René Enjilian

August 2, 2026

In this first milestone we explain the architecture of I-JEPA and highlight its design decisions. For this, we first give an overview of the general JEPA and related approaches like Joint-embedding and generative architectures. By comparing the other approaches, we also explain the motivation behind this novel approach. In the final sections, we discuss the dataset and our prototype implementation.

1 The World Model

In his position paper LeCun (2022), Yann LeCun outlines his vision for achieving more human-like artificial intelligence—systems that learn as humans do, capable of reasoning, planning complex task sequences, and driven by intrinsic objectives rather than external supervision or rewards. Current machine learning systems fall far short of human-level understanding and learning abilities. For example, while a human can learn to drive a car with just 20 hours of practice, today’s self-driving systems require massive amounts of supervised data and millions of reinforcement learning trials in simulation, yet still fail to match the proficiency of human drivers.

According to LeCun, the superior capabilities of humans and animals may stem from their ability to learn *world models*, i.e., internal representations of how the world works. These models allow humans to reason, plan, and predict efficiently, even in the absence of direct sensory input. At the core of LeCun (2022) is the Joint Embedding Predictive Architecture (JEPA), which provides a method to train predictive world models by learning a hierarchy of latent representations. To fully understand JEPA’s role and significance, it is first necessary to examine the broader cognitive architecture proposed by LeCun, from which JEPA is conceptually derived. Figure 1 depicts the overall system architecture from a cognitive perspective. It consists of a number of modules and we assume them to be differentiable, meaning any module connected to another can receive gradient estimates of the cost module’s scalar output with respect to its own output. In the following, we aim to briefly explain each component and its relationship to the others:

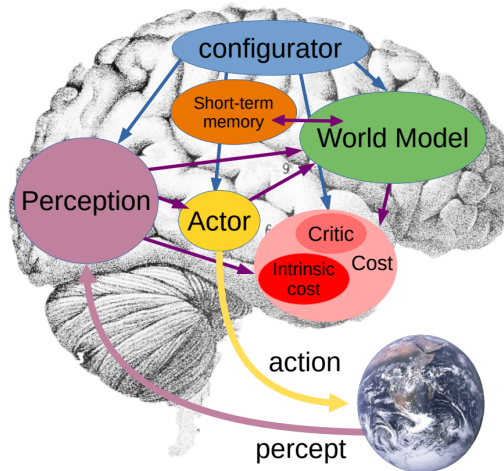


Figure 1: Cognitive system architecture for autonomous intelligence LeCun (2022).

- **Configurator Module:** The central controller of the system, it configures the perception, world model, cost, and actor modules based on the task. It adjusts their parameters to align them with the specific goal.
- **Perception Module:** Processes sensory input to estimate the current state of the world and represents it hierarchically with different levels of abstraction. It provides state information to the world model and actor, focusing on task-relevant details as directed by the configurator.
- **World Model Module:** Predicts missing information and future states of the world, either independently or based on actions proposed by the actor. It provides these predictions to the cost module for evaluation and helps the actor plan actions by simulating their outcomes. The configurator adjusts its focus to handle specific tasks or uncertainties.
- **Cost Module:** Evaluates the "energy" of current or predicted states, which represents the discomfort of the agent. This energy is computed as the sum of two sub-modules. The first is the *Intrinsic Cost module*, which is hardwired to compute immediate discomfort based on the current state (e.g., pain). The other one is the *Trainable Critic module*, which learns to predict future discomfort based on world model predictions and past data from memory.
- **Short-Term Memory Module:** Stores past, current, and predicted states along with their associated costs. It supports the world model in filling gaps, correcting errors, or making predictions, and allows the critic to learn from historical data.
- **Actor Module:** Proposes sequences of actions and evaluates them using predictions from the world model and costs from the cost module. It optimizes actions to minimize cost using gradients provided by the world model and cost. The configurator directs its behavior, while the actor outputs optimized actions to the system's effectors.

1.1 Principles for Training World Models

After defining the concept of the world model, the next step is to develop effective architectures and training paradigms for its implementation. Training the world model is a prototypical example of self-supervised learning (SSL), where the model predicts missing information and future states as forms of pattern completion. In SSL, the system is trained to capture the mutual dependencies between different parts of its input. Let x denote the observed part of the input and y the potentially unobserved part. Rather than predicting y directly from x , which can yield infinitely many solutions, the model only evaluates whether a proposed y is compatible with a given x . For instance, if x is a video snippet, y could represent a plausible (i.e., compatible) continuation of the video. There could be infinitely many reasonable ways to continue the video.

The framework of Energy-Based Models (EBMs) provides a general formulation for this concept LeCun et al. (2006). An EBM defines a scalar-valued function $F(x, y)$ that assigns low energy to compatible pairs (x, y) and higher energy to incompatible ones. This formulation allows the system to capture multi-modal dependencies, where multiple y values may be compatible with a given x . A latent variable z is often introduced to represent such dependencies. A latent variable is an inferred input variable that parameterizes the relationships between x and the set of compatible y . It encodes information about y that cannot be directly derived from x . For example, suppose x and y are photos of the same scene from slightly different viewpoints. In that case, the displacement of the camera, which must be inferred to verify compatibility, serves as the latent variable.

In the EBM formulation, a latent variable z is introduced to parameterize the energy function $E_w(x, y, z)$. When given a pair (x, y) the inference procedure of the EBM finds the value of z that minimizes the energy:

$$\tilde{z} = \arg \min_{z \in Z} E_w(x, y, z). \quad (1)$$

This latent-variable inference by minimization allows us to eliminate z from the energy function:

$$F_w(x, y) = \min_{z \in Z} E_w(x, y, z) = E_w(x, y, \tilde{z}) \quad (2)$$

Training an EBM involves constructing a deep neural network to compute the energy function $F_w(x, y)$, parameterized by a weight vector w , such that the trained w shapes the energy function correctly. For a given x in the training set, a well-trained $F_w(x, y)$ assigns lower energy to values of y associated with x and higher energy to other values of y . Training requires defining a loss function $L(x, y, F_w(x, y))$, which can also be expressed directly as $L(x, y, w)$. Minimizing this loss ensures that the energy $F_w(x, y)$ for the training sample (x, y) is lower than $F_w(x, \hat{y})$ for any $\hat{y} \neq y$.

The challenge is to ensure that the energy of $\hat{y}$ (values different from y) is higher than that of y , thereby avoiding representation collapse. Two main approaches to address this are contrastive methods and regularized methods. LeCun critiques contrastive methods LeCun (2022) due to the *curse of dimensionality*, where the number of negative samples (or comparisons) grows exponentially with the dimensionality of the representation Jing et al. (2021). In contrast, regularized methods avoid this issue by defining a loss function that both reduces the energy of training samples and minimizes the volume of the y space associated with low energy.

1.2 Joint Embedding Predictive Architecture

Having outlined the principles and requirements for designing architectures to train world models, we now turn to the Joint Embedding Predictive Architecture (JEPA), which emerges as a concrete realization of these concepts and serves as a practical framework for implementing predictive world models. Figure 2 shows a generic JEPA. It is designed to capture dependencies between two variables, x and y , without directly generating y from x . Instead, it operates in representation space, predicting an abstract representation of y from a representation of x . This is achieved through two encoding branches: one encoder computes $s_x = \text{Enc}_x(x)$, a representation of x , while the other computes $s_y = \text{Enc}_y(y)$, a representation of y . The encoders can differ in architecture and parameters, enabling flexibility in handling x and y of different modalities, such as video and audio.

A predictor module predicts s_y from s_x , potentially with the help of a latent variable z . The prediction error in representation space defines the energy function:

$$E_w(x, y, z) = D(s_y, \text{Pred}(s_x, z)) \quad (3)$$

The latent variable z is inferred by minimizing the energy:

$$\tilde{z} = \arg \min_{z \in Z} E_w(x, y, z) = \arg \min_{z \in Z} D(s_y, \text{Pred}(s_x, z)) \quad (4)$$

The overall energy is thus given by:

$$F_w(x, y) = \min_{z \in Z} E_w(x, y, z) = D(s_y, \text{Pred}(s_x, \tilde{z})) \quad (5)$$

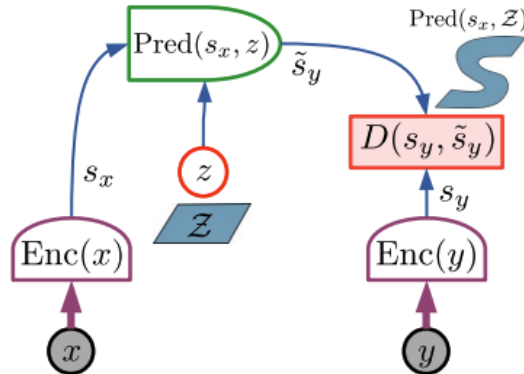


Figure 2: A generic JEPA LeCun (2022).

JEPA’s key advantage lies in performing predictions in representation space, avoiding the need to predict every detail of y . This allows the encoders to abstract away irrelevant details while focusing on task-relevant information. For instance, if x and y are video clips of a car before and after a fork in the road, the representations s_x and s_y might focus on attributes like the car’s position, orientation, and velocity, while ignoring irrelevant details like the texture of the sidewalk or surrounding trees. Here, the latent variable z could represent the car’s choice of turning left or right.

JEPA supports the representation of multiple plausible values of y compatible with x through two mechanisms: (1) the y -encoder $Enc_y(y)$ can exhibit invariance, producing the same representation s_y for a set of different y . This invariance ensures that the energy remains constant across this set, allowing the model to capture multi-modal dependencies; (2) the latent variable z , when varied over the possible set of latent variables Z , generates a range of plausible predictions, represented as $\{\tilde{s}_y = \text{Pred}(s_x, z) \forall z \in Z\}$. These predictions correspond to the diverse potential outcomes compatible with x .

2 Background

This section briefly discusses other common architectural approaches, specifically joint embedding and generative architectures, and highlights differences to JEPA. Like JEPA, these other two can also be described in the framework of EBMs.

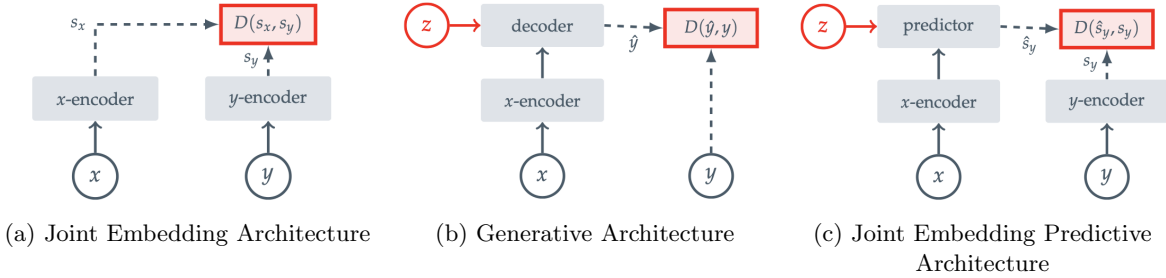


Figure 3: Comparison of the three architectures Assran et al. (2023).

2.1 Joint Embedding Architecture

Invariance-based pretraining can be framed as an Energy-Based Model (EBM) using a Joint-Embedding Architecture (JEA), which outputs similar embeddings for compatible inputs x and y and dissimilar embeddings for incompatible ones (see Figure 3a). For example, in image-based pretraining, x and y could be two augmented views of the same image generated by applying random, hand-crafted data augmentations Chen et al. (2020) such as cropping, rotation, or color jittering.

A significant challenge in JEAs is representation collapse, where the encoder produces constant outputs regardless of the input. This results in a flat energy landscape, where $F(x, y)$ fails to distinguish between compatible and incompatible pairs. In the example, collapse would mean the embeddings for all augmented views of an image x are the same, preventing the model of learning meaningful features.

Various strategies exist to avoid collapse. Contrastive losses explicitly push embeddings of incompatible pairs (negative examples) apart, ensuring meaningful separation Chen et al. (2020). Instead of relying on negatives, non-contrastive methods aim to reduce informational redundancy between embeddings by enforcing diversity Jing et al. (2021). Heuristic techniques introduce architectural constraints that naturally mitigate collapse. Such a heuristic would be the design of asymmetric architectures for the x -encoder and y -encoder Assran et al. (2023).

2.2 Generative Architecture

Reconstruction-based methods for self-supervised learning can be framed as Energy-Based Models (EBMs) using Generative Architectures (see Figure 3b). These architectures reconstruct a signal y from a compatible signal x using a decoder network,

which is conditioned on an additional variable z , often a latent variable, to facilitate reconstruction. In image-based pretraining, a common approach is to generate x, y pairs by masking parts of an image y to create x , where x is a copy of y with some patches masked. The variable z represents the masking pattern, often including positional tokens, and guides the decoder in reconstructing the masked patches. For example, y could be a complete image, x is the same image with 50% of its patches masked, and z is the specific tokens indicating which patches are missing.

Unlike Joint-Embedding Architectures, representation collapse is not a concern in these models as long as the informational capacity of z is significantly lower than that of the total signal y . For instance, in the example above, the informational capacity of z corresponds to the information needed to describe the position of the 50% masked patches, which is far less than the total information contained in the entire image y .

2.3 Comparison with JEPA

The Joint-Embedding Predictive Architecture combines the Joint-Embedding Architecture and the Latent-Variable Generative Architecture. As shown in Figure 3c, JEPAs share similarities with Generative Architectures in that they use a predictor network conditioned on an additional variable, z , to facilitate predictions. However, unlike Generative Architectures, JEPAs apply their loss function in embedding space rather than input space. This change in the loss function enables them to focus on abstract, task-relevant features rather than reconstructing raw data.

In contrast to Joint-Embedding Architectures, which aim for invariance to hand-crafted data augmentations, JEPAs seek embeddings that are predictive of each other when conditioned on additional information z . This property allows JEPAs to capture dependencies without requiring explicit invariance to input transformations. However, representation collapse remains a concern for JEPAs, as it is for JEAs. To address this, JEPAs leverage an asymmetric architecture between the x - and y -encoders, ensuring that the model learns meaningful and non-redundant embeddings.

3 I-JEPA

After defining the architecture and motivation of a generic JEPA, we now turn to Image-based Joint Embedding Predictive Architecture (I-JEPA). I-JEPA is an explicit realization of JEPA for image-based self-supervised learning Assran et al. (2023). Figure 4 gives an overview of the model components and their relationship.

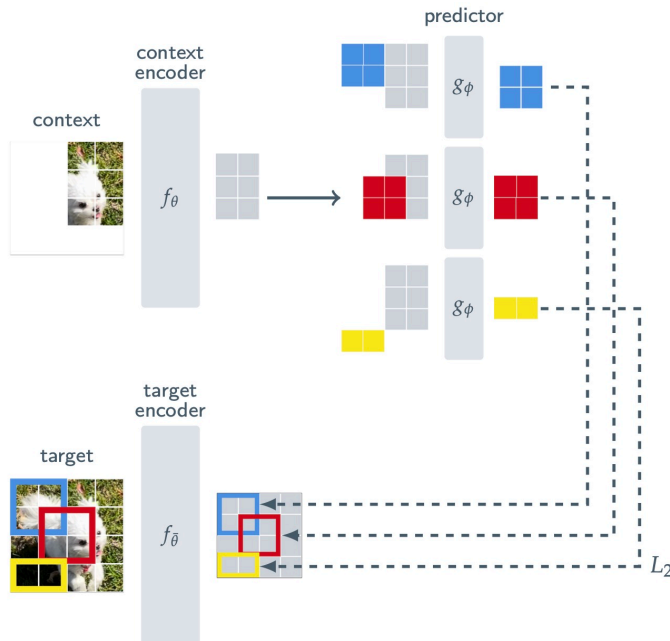


Figure 4: I-JEPA Assran et al. (2023).

I-JEPA predicts the representations of several target blocks within an image based on a single context block. The context encoder, implemented as a Vision Transformer (ViT) Dosovitskiy et al. (2020), processes only the visible patches from the context. A narrow ViT predictor then uses the output of the context encoder, along with positional tokens (depicted in color), to predict the representation of a specific target block. The target encoder produces the target embeddings, updated iteratively via an exponential moving average of the context encoder’s weights.

3.1 Model Components

Targets. The targets in I-JEPA correspond to the embeddings of image blocks. Figure 5 shows the process of generating embeddings. The input image y is converted into a sequence of N non-overlapping patches. The target-encoder $f_{\bar{\theta}}$, which is a Vision Transformer (ViT), produces a set of N embeddings $s_y = \{s_{y_1}, \dots, s_{y_N}\}$ (i.e., one for each patch) with s_{y_k} being the embedding of the k^{th} patch.

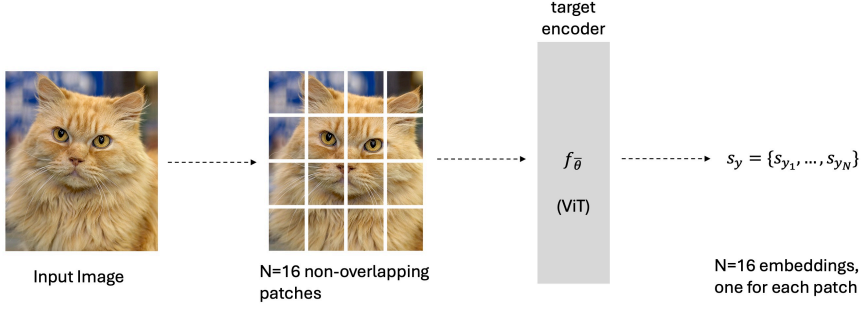


Figure 5: Generating patch-level representations.

To generate targets for the loss, M (potentially overlapping) blocks are randomly sampled from the target representations s_y , with each block represented as $s_y(i) = \{s_{y_j}\}_{j \in B_i}$, where B_i is the mask for the i^{th} block; see Figure 6. Generally, M is set to 4, and blocks are sampled with random aspect ratios between 0.75 and 1.5 and scales between 0.15 and 0.2. Notably, the target blocks are derived by masking the output of the target encoder, not its input, to ensure that the representations capture high-level semantics.

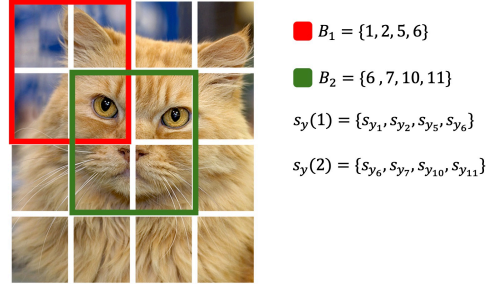


Figure 6: Target Blocks represented as sets of embeddings with the associated image patches. In this case, M equals 2 with B_1 and B_2 as the block masks and $s_y(1)$ and $s_y(2)$ as the corresponding target blocks. The two blocks overlap at embedding s_{y_6} .

Context. I-JEPA aims to predict target block representations from a single context block. The context block x is sampled from the image with a random scale between 0.85 and 1.0 and a unit aspect ratio. Its associated mask is denoted as B_x . Since target blocks are sampled independently, overlaps with the context block may occur. Removing overlapping regions from the context block ensures meaningful predictions. The resulting masked context block x is then processed by the context encoder f_{θ} , producing the patch-level representation $s_x = \{s_{x_j}\}_{j \in B_x}$. Figure 7 shows examples of context blocks and their corresponding targets.

Prediction. The prediction task in I-JEPA involves using the context encoder output, s_x , to estimate the M target block representations $s_y(1), \dots, s_y(M)$. For each target block $s_y(i)$, defined by its mask B_i , the predictor $g_\phi(\cdot, \cdot)$ takes s_x and a set of mask tokens $\{m_j\}_{j \in B_i}$ as input. The predictor outputs a patch-level prediction $\hat{s}_y(i) = \{\hat{s}_y(j)\}_{j \in B_i}$, corresponding to the target block. Mask tokens are shared learnable vectors enhanced with positional embeddings to encode spatial information. The prediction process is repeated M times, once for each target block, by conditioning the predictor on the corresponding mask tokens, resulting in predictions $\hat{s}_y(1), \dots, \hat{s}_y(M)$.

Loss. The loss is the average L_2 distance between the predicted patch-level representations $\hat{s}_y(i)$ and the target representations $s_y(i)$:

$$\frac{1}{M} \sum_{i=1}^M D(\hat{s}_y(i), s_y(i)) = \frac{1}{M} \sum_{i=1}^M \sum_{j \in B_i} \|\hat{s}_{y_j} - s_{y_j}\|_2^2 \quad (6)$$

The parameters of the predictor (ϕ) and the context encoder (θ) are optimized using gradient-based methods. Meanwhile, the target encoder parameters ($\bar{\theta}$) are updated via an exponential moving average (EMA) of the context encoder’s parameters. Using an EMA for the target encoder is critical for stable training, as shown in prior work with Vision Transformers Caron et al. (2021), and applies to I-JEPA, too.

4 ImageNet

The ImageNet dataset is a large-scale visual database extensively utilized in computer vision research. It comprises over 14 million labeled images spanning approximately 22,000 categories, structured according to the WordNet hierarchy Deng et al. (2009). A widely recognized subset, ImageNet-1K or ILSVRC (ImageNet Large Scale Visual Recognition Challenge), includes 1.2 million images divided into 1,000 classes. This subset has become the de facto standard benchmark for many computer vision tasks Russakovsky et al. (2015).

While ImageNet is a well-curated dataset, it has limitations. Some images suffer from noise, including motion blur, overexposure, and inadequate lighting. Additionally, many images lack isolated representations of the target class, instead depicting cluttered scenes, multiple objects, or ambiguous contexts. These imperfections create inconsistencies that pose additional challenges for model training, requiring robustness and generalization to learn meaningful representations effectively.

ImageNet is commonly used in supervised tasks such as image classification, object detection, and segmentation, serving as a benchmark for evaluating model performance. Beyond supervised learning, it has become central to self-supervised learning (SSL) and pretraining, where models learn representations directly from the data without requiring labeled examples, as with I-JEPA. Using ImageNet for pretraining in our project not only facilitates the use of pretrained components within our architecture but also ensures higher comparability with results from other research. The left column of Figure 7 shows some example images from ImageNet.

5 Baseline Method

This section describes our prototype implementation, including masking, encoders, and the predictor. We explain the training pipeline and evaluate the prototype.

5.1 Preprocessing and Masking Implementation

The preprocessing pipeline begins in our implementation by resizing all input images to a fixed resolution of 224×224 pixels. This step ensures uniformity across the dataset and aligns with the input requirements of Vision Transformer (ViT) architectures. The resized images are then divided into non-overlapping patches of size 16×16 pixels, resulting in a grid of 14×14 patches (i.e., $224/16 = 14$). This patch size is consistent with the ViT-B/16 architecture and aligns with the I-JEPA framework described in the literature Dosovitskiy et al. (2020); Assran et al. (2023).

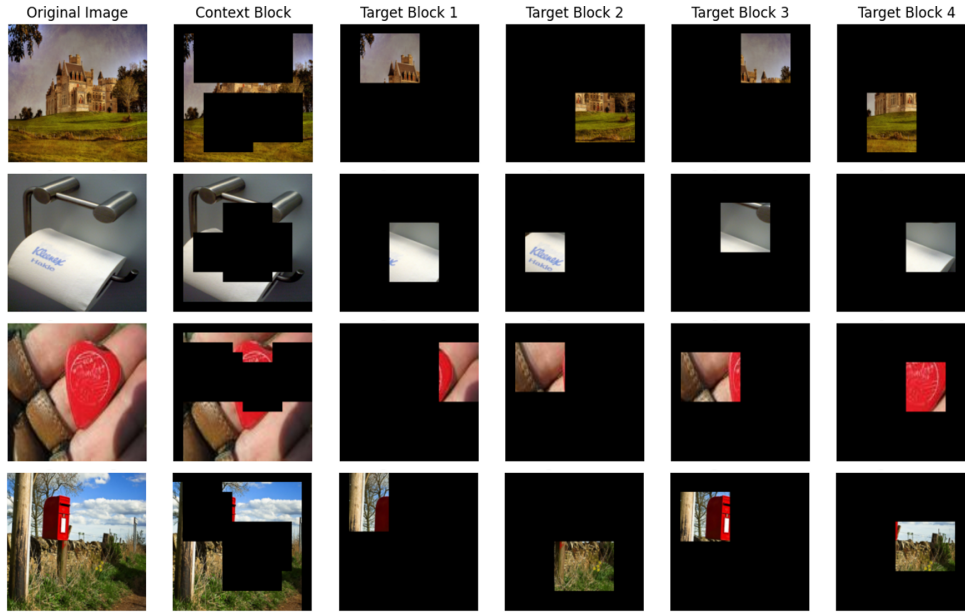


Figure 7: Four samples with the original images, the context block, and the four target blocks of the I-JEPA masking strategy taken from our implementation.

For the masking procedure, we operate directly in the patch space. We generate binary masks of size 14×14 for the context block and four distinct target blocks. The context mask identifies visible patches that the context encoder processes, while each target mask highlights the patches used as target regions for prediction by the model. Crucially, overlapping patches with the target block are masked in the context mask, ensuring no information leakage occurs between the context and target regions. This masking strategy closely follows the principles outlined in the I-JEPA framework. Figure 7 shows samples of the generated masks from our implementation with the ImageNet dataset.

5.2 Model Implementation: Encoders and Predictor

Our prototype utilizes distinct encoders for the context and target blocks, analogous to I-JEPA. Both encoders employ the Vision Transformer (ViT) architecture, and we initialize them with `Timm` library. The context encoder processes visible patches from the context block. In contrast, the target encoder generates embeddings for the entire image initially and subsequently saves the embeddings for the target regions (see asynchronous encoders in 3.1). Internally, each encoder includes patch-splitting functionality, with the input image divided into 16×16 patches and flattened into vectors of size $16 \times 16 \times 3 = 768$ (for RGB images). These vectors are passed through a learnable linear projection layer, transforming them into a 768-dimensional embedding space, which is the standard for the ViT-B/16 architecture. Positional embeddings are then added to these patch embeddings, allowing the transformer to encode spatial information and retain the structural layout of the image. Each 16×16 patch is treated as a "token," analogous to words in natural language processing tasks. Following this, multiple transformer layers consisting of self-attention and feed-forward operations process the patch embeddings, resulting in spatially-aware patch embeddings.

The paper describes the predictor network as a lightweight ViT. It does not need to process the raw image from scratch; it only takes the patch embeddings from the context encoder. Hence, we also build a narrow ViT for our predictor, reducing the number of transformer layers and self-attention heads to six each. The Predictor module in our implementation features a learnable mask token, which acts as an initialization for the embeddings of masked patches. During forward propagation, we add positional embeddings to the visible context embeddings and the mask tokens, which are then concatenated and processed through the lightweight Vision Transformer encoder with six layers.

5.3 Training Procedure

Our training pipeline integrates the above components into a cohesive framework. First, we preprocess the input image, generating the context and target masks. The masked patches are fed into the context encoder, producing embeddings for the visible patches. Simultaneously, the target encoder processes the target regions to generate the "ground-truth" target embeddings. The predictor takes the context encoder's output and learns to predict the embeddings of the target blocks by attending to the spatial structure of the masked regions. During training, we compute the predictor's loss as the L_2 distance between its predictions and the ground-truth embeddings from the target encoder. This loss is then backpropagated through the context encoder and predictor, while the target encoder's weights are updated via the exponential moving average (EMA) of the context encoder weights. Over multiple training epochs, the model learns to generate representations predictive of masked regions in representation space, capturing meaningful features from the input images.

5.4 Results and Evaluation

Our implementation serves as a prototype to validate the functionality and logic of I-JEPA. The masking strategy, a core component of the I-JEPA architecture, has been fully implemented, including a visualization mechanism to inspect both context and target masks (see samples in Figure 7). The training loop for the prototype is now executable, enabling us to evaluate its behavior in the following milestone.

The prototype has not undergone extensive training at this stage, and downstream tasks have not yet been implemented. Consequently, we cannot provide quantitative results or assess the validity of the learned representations. Instead, our current focus is on ensuring the correctness of the masking, encoding, and prediction mechanisms and verifying that the training pipeline functions as intended. Future steps will involve training the model on a larger dataset and conducting downstream task evaluations to validate the quality of the learned representations.

6 Discussion

This first milestone focused on laying the theoretical and practical groundwork for understanding and implementing I-JEPA. The theoretical exploration highlighted how I-JEPA differs from traditional self-supervised learning methods by predicting embeddings in abstract representation spaces, rather than reconstructing pixel or token-level details. This approach aims to produce semantic-level representations that are highly applicable across a broad spectrum of downstream tasks. For example, unlike pixel-level methods such as Masked Autoencoders (MAE) He et al. (2022), I-JEPA leverages abstract predictions in embedding space, reducing reliance on unnecessary pixel-level details while learning more semantically rich features.

From a practical standpoint, we successfully implemented the masking strategy, a core component of I-JEPA, and generated sample patches using ImageNet data. Additionally, we prototyped a basic model architecture using Vision Transformers for the Context and Target Encoders and the Predictor module. These steps are essential for scaling the project towards pretraining and evaluation.

6.1 Reflection on Our Work

The I-JEPA model stack, encompassing preprocessing, patch creation, embedding transformations, and an architecture with three ViTs, is complex and required a thorough understanding. We began by discussing the architecture's components—encoder, predictor, and masking strategies—ensuring clarity on its theoretical basis. This enabled us to identify its distinctions and advantages over previous approaches, such as invariance-based methods (e.g., SimCLR Chen et al. (2020)) and pixel-reconstruction models (e.g., MAE He et al. (2022)), highlighting its scalability and semantic representation capabilities. Adopting an iterative approach, we first implemented the masking strategy as a foundational component. Then, incrementally developed the context and target encoders and the lightweight predictor, aligning each module with the pretraining framework.

6.2 Outlook for the Next Milestone

The upcoming milestone will focus on solidifying the model implementation and initiating pretraining the Vision Transformers from scratch. Our goals include:

1. **Pretraining the I-JEPA Model:** This step involves scaling our prototype into a fully operational system capable of generating meaningful embeddings from ImageNet samples.
2. **Refinement of the Prototype:** As we iteratively develop the prototype, we focus on refining its components, including the encoders, predictor, and training loop aligning with the theoretical foundations of I-JEPA.
3. **Defining Evaluation Metrics:** Evaluating self-supervised models like I-JEPA can be challenging since traditional supervised metrics like accuracy do not apply directly. We will explore metrics such as linear probing and fine-tuning performance, which measure how well the pretrained embeddings transfer to downstream tasks.
4. **Evaluation through Downstream Tasks:** By training lightweight classifiers on frozen representations (linear probing) or fine-tuning the entire model, we will assess the semantic quality and versatility of the learned representations.

References

- M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023.
- M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.
- T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020.
- J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022.
- L. Jing, P. Vincent, Y. LeCun, and Y. Tian. Understanding dimensional collapse in contrastive self-supervised learning. *arXiv preprint arXiv:2110.09348*, 2021.
- Y. LeCun. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.
- Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, F. Huang, et al. A tutorial on energy-based learning. *Predicting structured data*, 1(0), 2006.
- O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115:211–252, 2015.

Project Machine Learning

— Milestone 2 —

René Enjilian

August 2, 2026

We evaluate the Image-based Joint Embedding Predictive Architecture (I-JEPA) in the second milestone. We first briefly recap I-JEPA’s architecture and explain the training regime and its parameters. In the second section, we turn to the method’s evaluation, exploring the embedding characteristics.

1 Implementation

This section briefly summarizes I-JEPA’s architecture and notes the distinct components employed in the subsequent evaluation. We then justify the selected hyperparameters for training and examine the loss curve, reflecting on the convergence.

1.1 I-JEPA

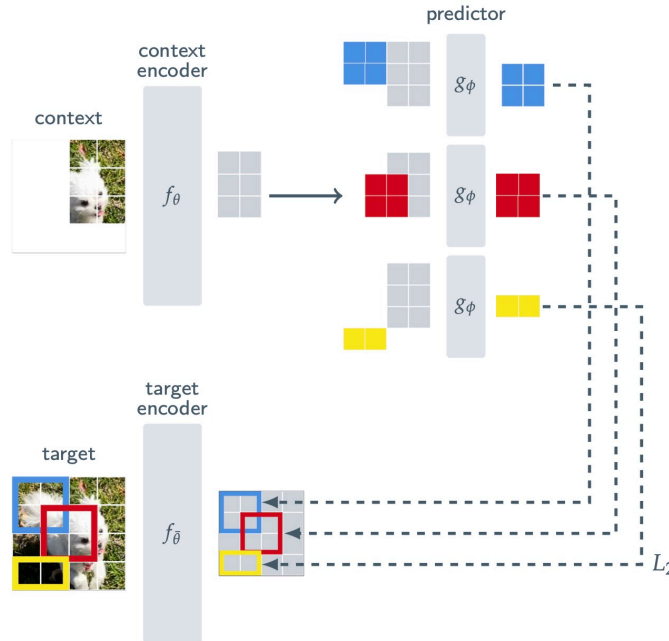


Figure 1: I-JEPA Assran et al. (2023).

I-JEPA Assran et al. (2023) is a joint-embedding predictive architecture LeCun (2022) that uses a single context block from an image to predict the representations of multiple target blocks drawn from the same image. As shown in Figure 1, it comprises a Vision Transformer (ViT) context encoder, which processes only the visible patches within the context block. A narrower ViT predictor then uses the context encoder’s outputs and positional tokens to infer the representation for each target block. A separate target encoder generates the final target representations, whose weights are updated at every iteration through an exponential moving average of the context encoder’s parameters. The target encoder is also a Vision Transformer, sharing the same architecture as the context encoder.

1.2 Training

We outline our approach to training the I-JEPA model, focusing on the primary hyperparameters and resource setup. We employ a Vision Transformer (ViT)-base as both the context encoder and target encoder, while the predictor is a shallower ViT with a depth of 12 layers and 384-dimensional embeddings. All components operate in bfloat16 precision. To manage training efficiently, we distribute the workload across 8 NVIDIA A100 GPUs, each processing a mini-batch of size 128.

Our training schedule spans 600 epochs, beginning with a low learning rate of 0.0002 that linearly warms up to 0.001 over 40 epochs. After reaching the peak learning rate, it gradually decays to a final value of 1.0e-6 by the end of training. An exponential moving average (EMA) of the model parameters is also maintained, starting at 0.996 and annealing to 1.0. We apply weight decay in two phases: an initial weight decay of 0.04 followed by a final weight decay of 0.4.

We also apply a masking strategy characteristic of I-JEPA, setting different mask sizes and scales for the encoder and prediction targets to encourage the model to learn robust, context-aware representations. Table 1 summarizes the key hyperparameters—including optimizer settings and masking details—for quick reference. As in the original paper, we conducted pretraining on ImageNet1k. Given the setting and configuration described, the training took almost six days.

Table 1: Summary of Hyperparameters.

Parameter	Value
<i>Model Architecture</i>	
Context/Target Encoder	ViT-base
Predictor	Shallow ViT, depth = 12, emb. dim = 384
Precision	bfloat16
<i>Training Setup</i>	
GPUs	8 (NVIDIA A100 80GB)
Batch Size (per GPU)	128
Epochs	600
EMA	0.996 $\rightarrow$ 1.0
<i>Learning Rate Schedule</i>	
Start LR	0.0002
Peak LR	0.001
Warmup (epochs)	40
Final LR	1.0e-6
<i>Regularization</i>	
Weight Decay	0.04 $\rightarrow$ 0.4
<i>Mask Configuration</i>	
Encoder Mask Scale	[0.85, 1.0]
Prediction Mask Scale	[0.15, 0.2]
Patch Size	16
Number of Encoder Masks	1
Number of Prediction Masks	4

Loss. Figure 2 shows the average training loss over 600 epochs for our I-JEPA model with ViT-Base. Following the training protocol from I-JEPA Assran et al. (2023), we chose 600 epochs for ViT-Base—double the 300 epochs used for ViT-Huge—because smaller architectures typically need extended training for the loss to converge. Indeed, as illustrated in Figure 2, most of the loss reduction happens in later epochs (beyond about 450). The loss increases as expected during the first 40 epochs (warm-up). Afterward, it displays the typical JEPA pattern observed in related work such as T-JEPATHimonier et al. (2024), aligning with the stepwise behavior commonly reported in self-supervised learning Simon et al. (2023).

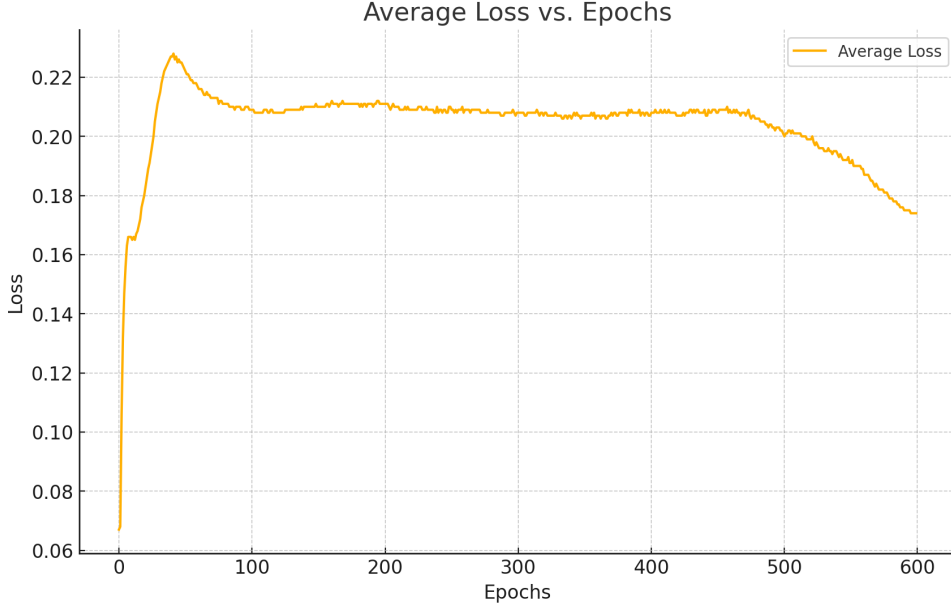


Figure 2: Training-loss over epochs.

2 Evaluation

In this section, we evaluate our trained I-JEPA model through experiments to examine its learned representations’ quality, organization, and stability. First, we compute cosine similarities between image embeddings to quantify how well the model aligns conceptually related content. Next, we visualize the embedding space to see if semantically similar images cluster together in lower-dimensional projections. Finally, we investigate embedding stability by measuring how invariant these representations remain under various image augmentations. These experiments offer a solid view of our model’s representational capabilities and reveal insights into its robustness and capacity for semantic encoding.

2.1 Cosine Similarity

Cosine similarity is a popular metric for measuring how similar two vectors are by calculating the cosine of the angle between them. In the context of learned embeddings—such as those produced by image encoders—it provides a straightforward way to assess semantic or conceptual closeness. Two embedding vectors that point in roughly the same direction have a high cosine similarity, indicating a strong correspondence in their underlying features. Formally, given two embedding vectors $\mathbf{u}$ and $\mathbf{v}$, the cosine similarity is defined as:

$$\text{cosine similarity} = S_C(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}, \quad (1)$$

where $\mathbf{u} \cdot \mathbf{v}$ denotes the dot product, and $\|\mathbf{u}\|$ and $\|\mathbf{v}\|$ represent Euclidean norms. The cosine similarity is defined on the interval $[1, -1]$. A value of 1 indicates the embedding vectors align in the same direction, -1 indicates they align in opposite directions, and zero indicates they are orthogonal. When embeddings from the same class have a high cosine similarity, the model has learned semantically meaningful representations. Alternatively, it can also hint at anomalies in the embedding space.

In the context of I-JEPA, we conduct cosine similarity comparisons between image embeddings produced by the target encoder. We utilize the target encoder because it holds the most up-to-date weights (i.e., EMA of context encoder). Hence, all inference and downstream tasks generally rely on the target encoder rather than the context encoder. Figure 3 visualizes the cosine similarities for 50 images (5 classes, each with 10 images). We carefully selected these images to incorporate challenging angles, surrounding scenes, and lighting. The five selected classes from ImageNet1k are dog (n02099712), cat (n02123045), car (n03770679), airplane (n02690373), and

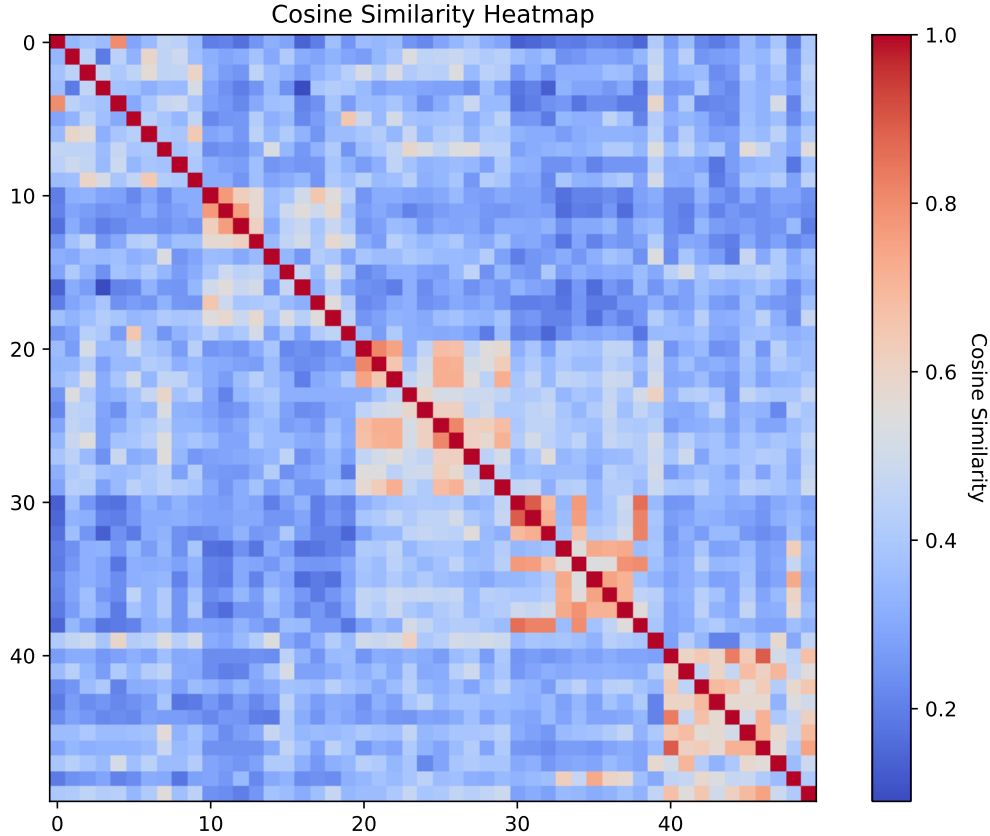


Figure 3: Cosine Similarity Heatmap. The Heatmap shows cosine similarities for 5 classes, each with 10 images. The classes are dog, cat, car, airplane, and flower. Positions 0 to 9 correspond to the dog image embeddings, 10 to 19 to cat, 20 to 29 to car, 30 to 39 to airplane, and 40 to 49 to the flower class.

flower (n11939491). We compute the cosine similarity for each image embedding to the other 49 embeddings, resulting in a 50×50 symmetric similarity matrix. For better illustration, we visualize the computed matrix as a heatmap with red indicating a high similarity between two embeddings and blue low similarity.

Discussion. Clearly, the values on the diagonal have a cosine similarity of 1 (dark red) since the cosine similarity of a vector to itself is always 1. Further, we observe that the square areas (10×10) around the diagonal are reddish (i.e., higher similarities), showing that embeddings from the same class have higher cosine similarities and thus indicating that the model learned semantic representations. A clear semantic pattern is visible while we only utilize a limited number of classes and images.

The cosine similarity heatmap is also helpful in identifying interesting outliers. Figure 4 highlights image pairs from different classes showing a moderately high cosine similarity. In the first example, the model correlates the dog with the animal shown on the aircraft’s tail. In the second example, the target encoder assigns a high cosine similarity to an aircraft and a flower, most likely due to both images’ deep blue sky. The third example shows that the model struggles with grayscale images. However, it should be noted that we trained our I-JEPA model only on ViT-Base given compute resource constraints. Further, training for more than 600 epochs might result in an even more robust model. Overall, the model assigns higher cosine similarities to image embeddings from the same class. This behavior indicates that it learned semantically meaningful image representations. Applying explainable artificial intelligence methods Adadi and Berrada (2018) might be interesting in later stages of the project when training I-JEPA on ViT-Huge to see if such confusions as shown in Figure 4 still occur and which parts of the image the trained target encoder uses for classification.

Position in Heatmap	Images		Cosine Similarity
(4, 39)			0.62
(35, 48)			0.73
(19, 5)			0.65

Figure 4: Image pairs from distinct classes with relatively high cosine similarity.

2.2 Visualizing the Embedding Space

Visualizing the embedding space is an effective way to gain intuitive insights into how a model organizes and clusters data. Because learned embeddings typically reside in high-dimensional spaces, techniques such as t-SNE, UMAP, or PCA project these embeddings onto a two- or three-dimensional plane. This low-dimensional representation can then be plotted, allowing one to observe potential clusters, groups, or outliers that share semantic similarities.

Such visualization not only helps in diagnosing the quality of embeddings but also in uncovering how different samples relate to each other. For example, items clustered closely often share meaningful features, while isolated points may flag anomalies or minority classes. However, it is important to note that these projections can introduce distortions. The chosen algorithm and parameters (e.g., perplexity in t-SNE) can significantly affect the resulting distribution, potentially highlighting or obscuring different aspects of the data.

We took a more microscopic view in the previous experiment by manually selecting the image samples. Here, we want to take a more macroscopic view by plotting the embedding space for more random samples. Figure 5 shows the plotted embedding space for 10 different classes with 100 random images each (i.e., 1000). In addition to the classes introduced in 2.1, we add guitar (n02676566), clock (n02708093), banana (n07753592), umbrella (n04507155), and church (n03028079) from ImageNet1k. Again, we utilize the target encoder with frozen weights (600 epochs) to produce the embeddings. ViT-Base produces 768-dimensional embeddings. We apply dimensionality reduction with t-SNE Van der Maaten and Hinton (2008) to capture the nonlinear structure of the embedding space in two dimensions. We assign separate colors to each class for better differentiation. If the model has learned meaningful semantic representations, the different class embeddings should form well-distinguishable clusters in the lower-dimensional visualization, granted that the dimensionality reduction algorithm can capture the embedding’s nonlinearity.

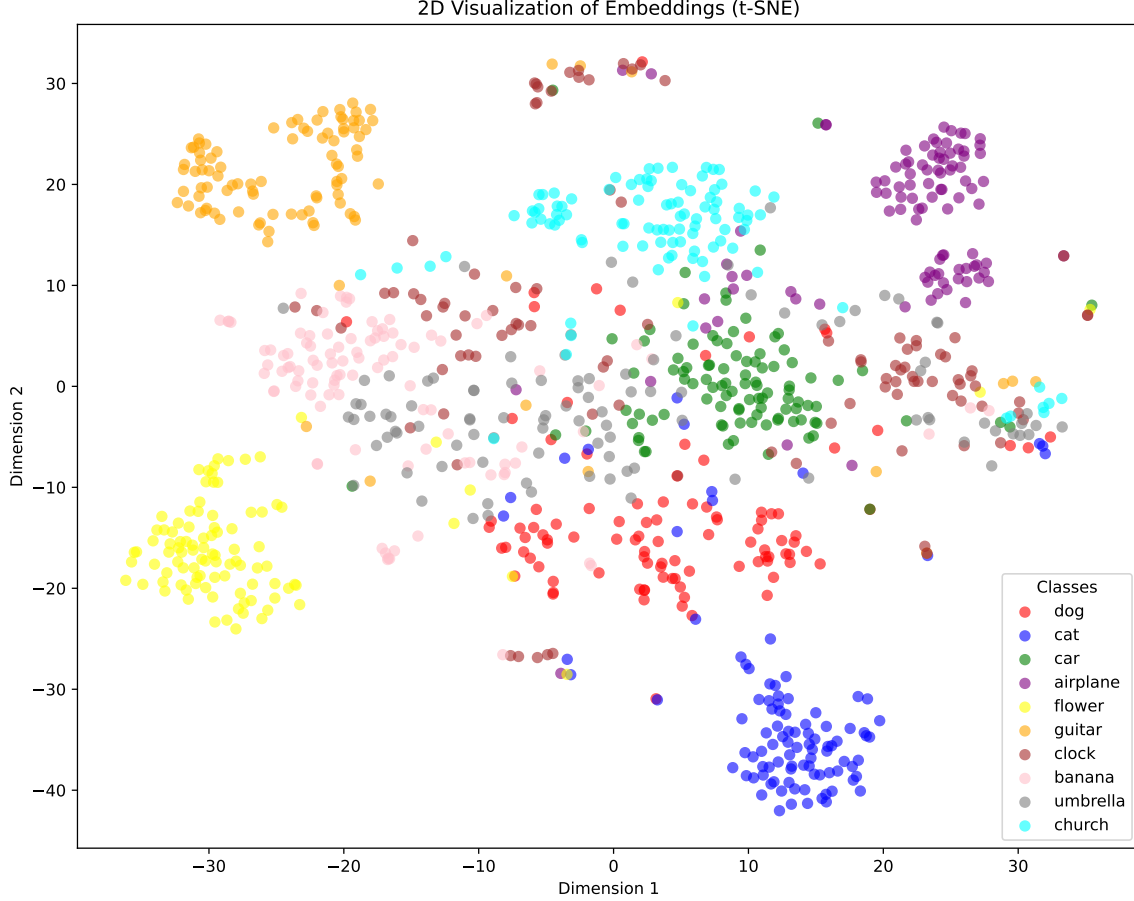


Figure 5: Embedding Space visualized in two dimensions.

Discussion. Figure 5 shows that the different class embeddings form well-separable clusters, indicating that the target-encoder learned robust representations and that t-SNE can capture the nonlinear structure of the embedding space. Certain categories, such as guitars, cats, airplanes, and flowers, create distinct clusters that are clearly separated from one another, with only a few outliers. The center appears more crowded, but the different clusters are still distinguishable. Overall, we can see that the model has learned semantically meaningful representations. We assume that a larger Vision Transformer trained on more epochs might also improve the clustering quality for the more crowded areas.

2.3 Embedding Stability over Augmentations

In his positional paper, Yann LeCun criticizes the current state of deep learning architectures LeCun (2022). He argues against contrastive learning techniques that rely on hand-crafted data augmentations. Methods like SimCLR Chen et al. (2020), DINO Caron et al. (2021), or iBOT Zhou et al. (2021) utilize augmentations to learn more robust features. However, this approach is computationally inefficient and prone to representation collapse. He is also critical of Generative Architectures that perform pixel reconstruction by applying the loss in the output space. Applying the loss in the output space shifts the focus from abstract, task-relevant features to reconstructing raw data (e.g., Masked Autoencoders He et al. (2022)). Hence, according to him, Generative architectures should be replaced with Joint Embedding Architectures. Combining all these considerations, he introduced JEPA, which applies the loss function in the embedding space. JEPA and its instantiations, like I-JEPA, seek embeddings that are predictive of each other, as shown in Figure 1 for I-JEPA, avoiding additional data augmentation. Thus, JEPA models aim to learn invariant embeddings, meaning the representations remain similar despite different augmentations since they focus on abstract, task-relevant features.

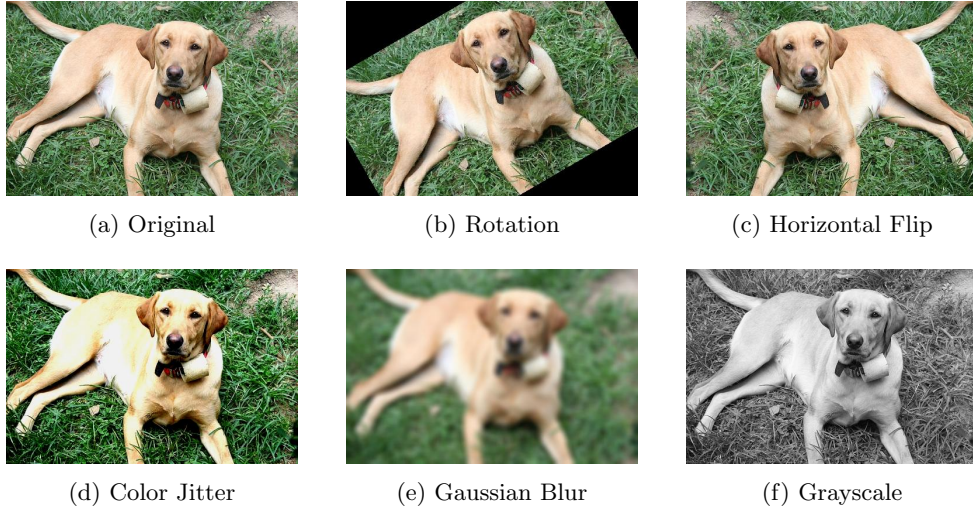


Figure 6: Original image and five augmented versions.

In this experiment, we want to test I-JEPA’s embedding invariance. We randomly select 1000 ImageNet1k images from 100 distinct classes (i.e., 10 images per class). For each image, we create five augmentations: rotation by 30° , horizontal flip, color jitter, Gaussian blur, and grayscale conversion. Figure 6 shows the different views for an example image. These transformations with the selected parameters are common in many SSL pipelines. The target encoder with frozen weights computes each image’s embeddings and five views. Afterward, we compute the cosine similarity between the original and each view’s embedding. If I-JEPA’s embeddings are indeed predictive of each other, the cosine similarities should be very high, although it does not use hand-crafted augmentations like contrastive methods.

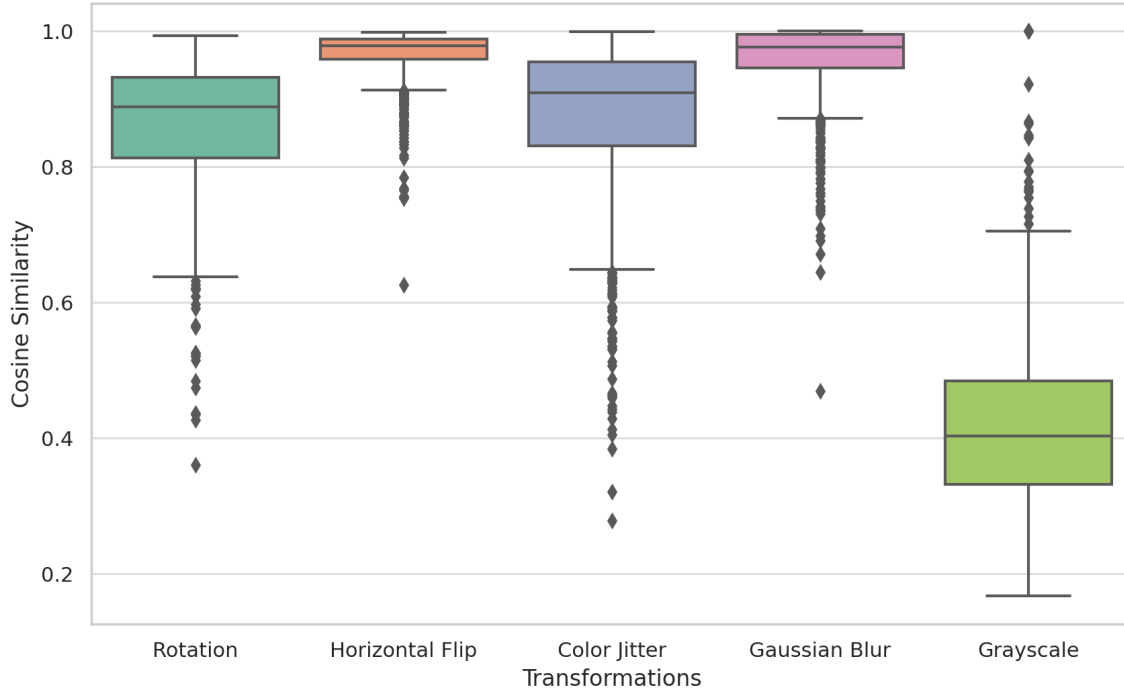


Figure 7: Boxplot of Cosine Similarities Across Transformations. Each box demonstrates how the transformed embeddings compare the original. A high median indicates strong similarity and thus strong embedding invariance. Color jittering uses brightness=0.4, contrast=0.4, saturation=0.4, and hue=0.1; Gaussian blur applies a 3×3 kernel.

Discussion. Figure 7 presents a box plot illustrating the distribution of cosine similarity scores across five image transformations: Rotation, Horizontal Flip, Color Jitter, Gaussian Blur, and Grayscale. The x-axis lists these transformations, while the y-axis indicates how similar the transformed embeddings are to the originals. Higher values suggest a greater invariance.

Rotation achieves a median similarity of approximately 0.90, with an interquartile range (IQR) from 0.81 to 0.93. These values show that most images retain high similarity under moderate rotation, but certain orientation-dependent features still introduce variability. Horizontal Flip has the highest median similarity, around 0.98, and an IQR of 0.96–0.99. This narrow range indicates that flipping has little to no effect on how the model encodes most images, suggesting near-complete invariance to horizontal reflections. Color Jitter displays a median of about 0.92, with an IQR spanning 0.82–0.96. Although all images undergo the same brightness, contrast, and saturation adjustments, the broader range indicates that specific images (perhaps those with particular color profiles) experience more significant representation shifts than others. Gaussian Blur shows a median near 0.97, with an IQR of 0.95–1.00, indicating that mild blurring leaves most embeddings virtually unchanged. Blurring reduces fine-grained details but leaves most of the semantic information intact. This result highlights the model’s stability when fine details are softened or partially removed. Grayscale stands out with a markedly lower median of roughly 0.40 and an IQR of 0.35–0.48, reflecting a dramatic reduction in similarity. This confirms our speculation from Figure 4 that I-JEPA might struggle with grayscale images. This result implies that removing color cues disrupts I-JEPA’s learned representations more than any other transformation tested. All transformations exhibit a moderate or low number of outliers, but Grayscale stands out by having only positive outliers. These are likely images that were already grayscale in their original form. In Figure 8, a boxplot with more stringent parameter settings for Gaussian Blur and Color Jitter is presented as a stress test, revealing a median similarity drop to 0.60 for Color Jitter and 0.55 for Gaussian Blur, thereby indicating significantly reduced embedding invariance.

These observations highlight that I-JEPA’s representations exhibit robust invariance to spatial transformations and mild blur, yet they remain substantially sensitive to removing color cues. The more stringent augmentation settings confirm that extreme color or blur distortions can significantly degrade the learned embeddings. A model trained with a larger backbone, such as ViT-Huge, may better capture nuanced visual information and thus maintain higher similarity under severe transformations. Overall, the results emphasize I-JEPA’s strong performance on moderate augmentations while pointing to color dependence as a key area for further improvement.

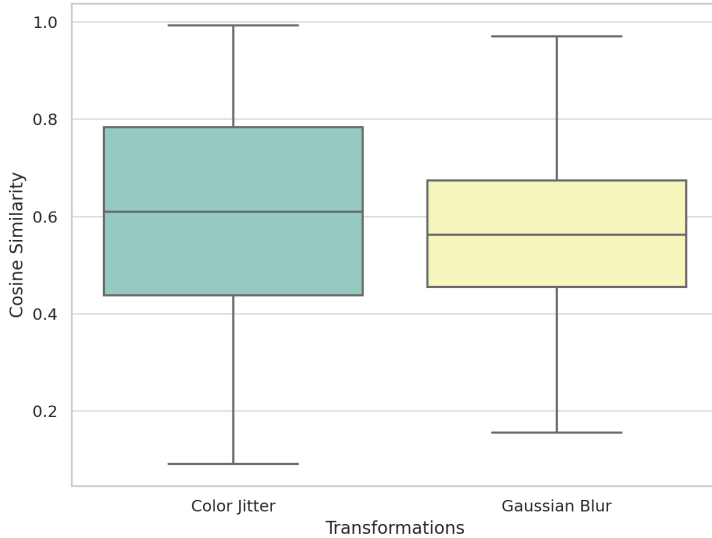


Figure 8: Boxplot of Cosine Similarities Across Transformations. Color jittering uses brightness=0.8, contrast=0.8, saturation=0.8, and hue=0.2; Gaussian blur applies a 15×15 kernel. This setting serves as a stress test.

References

- A. Adadi and M. Berrada. Peeking inside the black-box: a survey on explainable artificial intelligence (xai). *IEEE access*, 6:52138–52160, 2018.
- M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023.
- M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.
- T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020.
- K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16000–16009, 2022.
- Y. LeCun. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.
- J. B. Simon, M. Knutins, L. Ziyin, D. Geisz, A. J. Fetterman, and J. Albrecht. On the stepwise nature of self-supervised learning. In *International Conference on Machine Learning*, pages 31852–31876. PMLR, 2023.
- H. Thimonier, J. L. D. M. Costa, F. Popineau, A. Rimmel, and B.-L. Doan. T-jepa: Augmentation-free self-supervised learning for tabular data. *arXiv preprint arXiv:2410.05016*, 2024.
- L. Van der Maaten and G. Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.
- J. Zhou, C. Wei, H. Wang, W. Shen, C. Xie, A. Yuille, and T. Kong. ibot: Image bert pre-training with online tokenizer. *arXiv preprint arXiv:2111.07832*, 2021.

Project Machine Learning

— Milestone 3 —

René Enjilian

August 2, 2026

We continue the evaluation of the Image-based Joint Embedding Predictive Architecture. In the previous milestone, we evaluated the model’s output, that is, the embedding space and its characteristics. In this milestone, we focus on how the model makes its decisions. Since the target encoder is responsible for inference, we first explain the Vision Transformer before discussing the explainability experiments.

1 Vision Transformer

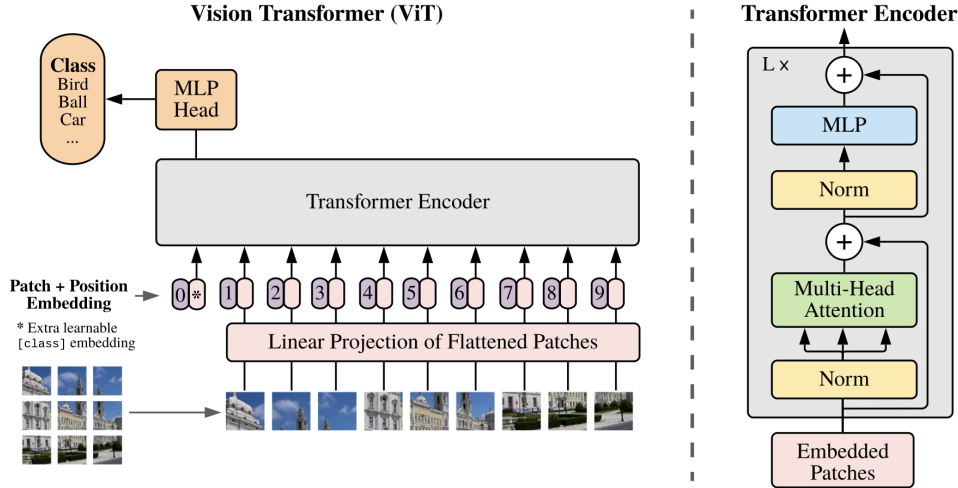


Figure 1: An overview of the Vision Transformer (ViT) architecture Dosovitskiy et al. (2020). The left side depicts the overall pipeline, which splits the input image into patches, embeds each patch, and then classifies the sequence through a stack of Transformer blocks. The right side shows a single Transformer encoder block in detail, consisting of a multi-head self-attention module and a feed-forward network.

Figure 1 presents the Vision Transformer (ViT) Dosovitskiy et al. (2020) pipeline in two parts. On the left, the pipeline partitions the input image into fixed-size patches (e.g., 16×16 pixels). Each patch passes through a linear projection, creating a corresponding patch embedding. A learnable class token then joins the sequence of patch embeddings, and positional embeddings ensure that spatial order remains intact. This sequence flows through a stack of Transformer encoder blocks before reaching an Multi-Layer Perceptron (MLP) classifier at the output.

On the right, the figure zooms in on a single Transformer encoder block. Each block includes two main sub-layers: a Multi-Head Self-Attention (MHSA) module and an MLP-based feed-forward network. Residual connections and layer normalization envelop both sub-layers, which stabilizes training and prevents vanishing gradients. By attending to every patch in the sequence, MHSA captures global relationships, and the feed-forward network refines these learned features. This combination enables ViT to build robust representations for diverse image tasks, leveraging the expressive power of self-attention to integrate information across the entire image.

2 Explainable Artificial Intelligence

In Milestone 2, we analyzed the learned embeddings by measuring cosine similarity, visualizing the embedding space, and assessing the stability of representations under various augmentations. These methods offered preliminary evidence of semantic clustering and robustness, but they provided limited insight into *why* certain features or regions influenced the model’s outputs. Milestone 3 extends these findings through explainable AI techniques that clarify I-JEPA’s internal mechanisms. We aim to examine how the model prioritizes different regions or concepts within images. By integrating these interpretability methods, we seek to validate and deepen our earlier results, revealing the specific factors that underpin the model’s performance.

2.1 Occlusion Analysis

Occlusion is an interpretability technique that systematically removes, or “occludes,” parts of an input image to assess their importance for a model’s learned representation Zeiler and Fergus (2014). Instead of examining internal weights or self-attention mechanisms, occlusion focuses on how much the final embedding changes when a region is missing. Researchers often divide the image into a grid and replace one patch at a time with a mask, such as a black square, the patch’s average color, or a blurred version of that patch. This approach aims to highlight regions whose removal causes significant shifts in the model’s embedding.

We compute an embedding for the original image and another embedding for the occluded version, then measure the difference as:

$$\Delta = 1 - \text{cos_sim}(e_{\text{orig}}, e_{\text{occl}}) \quad (1)$$

where $\text{cos_sim}(\cdot, \cdot)$ is the cosine similarity between the two embeddings:

$$\text{cos_sim} = S_C(e_{\text{orig}}, e_{\text{occl}}) = \frac{e_{\text{orig}} \cdot e_{\text{occl}}}{\|e_{\text{orig}}\| \|e_{\text{occl}}\|} \quad (2)$$

Larger Δ values indicate a more significant drop in similarity, implying higher importance for the occluded patch. This process produces a heatmap: each patch receives a score proportional to Δ , which can be visualized by overlaying warm colors for critical areas and cooler tones for less impactful regions. In contrast, attention maps from Vision Transformers Xu (2015) display the model’s internal focus at a particular layer rather than an explicit measure of how much each region influences the final embedding. Occlusion thus offers a direct way to infer which parts of the image the model relies on most, and varying the masking strategy can mitigate artifacts associated with high-contrast occlusions Fong and Vedaldi (2017).

This experiment builds on top of the second milestone’s first experiment. We compared 50 images from five distinct classes by their cosine similarity. Figure 8 shows the heatmaps for all 50 experimental images. We create patches of size 28×28 , resulting in an 8×8 grid. Then, we systematically occlude each patch and compute the impact on the embedding as described in equation 1. We utilize I-JEPA’s target encoder (ViT) to compute the embeddings. Due to the out-of-distribution artifacts introduced by black occlusion masks, we opt for a mask that averages the color in that patch. In milestone 2, we identified several images that exhibited low cosine similarity with images from their class or higher similarity with class-different images. Thus, we investigate the potential reasons with heatmaps generated by occlusion.

Figure 2 shows four challenging images with their respective class names. Although it represents an airplane, figure 2a has higher cosine similarity with the dog images. We observe that the animal printed on the airplane’s back has the greatest impact on the embedding and that the rest of the plane does not influence the embedding. Figure 2b has low cosine similarity with the other car-image embeddings. This image is challenging due to the pronounced position of the woman next to the car. We see that the car does not influence the embedding and that most attention goes to the woman and the lawn next to her. We also trained a lightweight classification head on the target encoder embeddings and observed that the encoder did not correctly classify the image as a car. Figure 2c shows a cat wrapped in a blanket with only the face sticking out. This image has very low cosine similarity

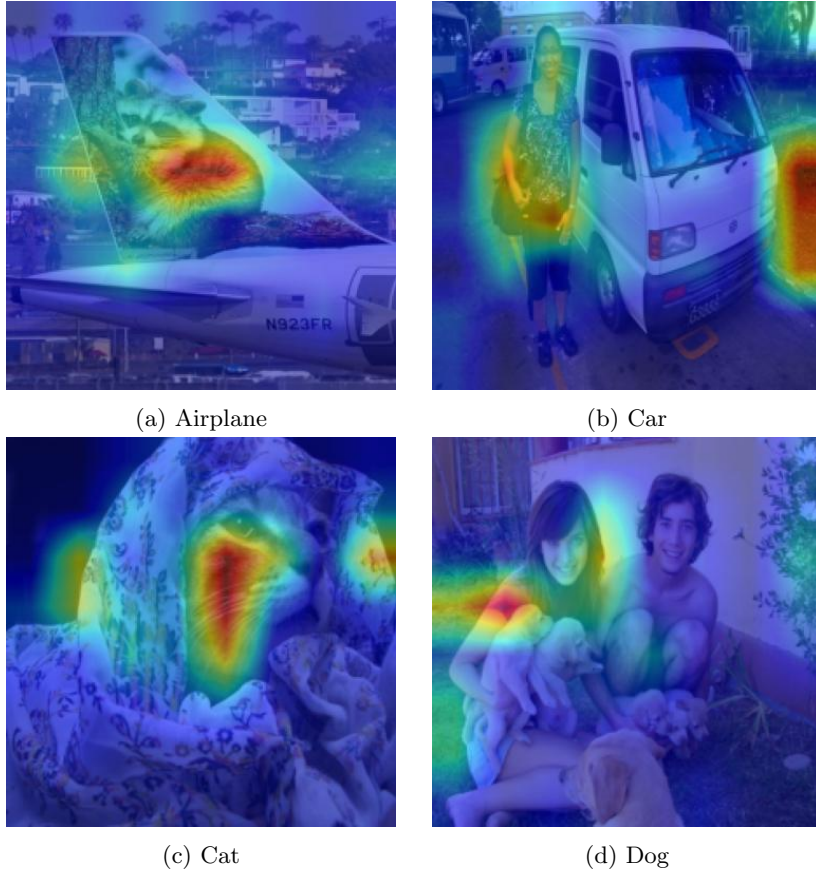


Figure 2: Images misclassified by the target encoder. The heatmap highlights the regions where the Vision Transformer primarily focused, clarifying the misclassification. Each patch measures 28×28 pixels, forming an 8×8 occlusion grid.

with the other cat images. We see that only the lower left side of the face influences the embedding and that prominent parts of the face are not taken into account by the ViT. It appears as if the target encoder confuses the highlighted part of the face with the blanket. Figure 2d shows an image from the dog class with very low cosine similarity to the other images of its class. According to the heatmap, the dogs in the image barely influence the embedding. Most of the impact comes from the woman on the left. Our classification head also did not correctly classify this image.

We also observe that I-JEPA sometimes relies more on the object’s contours than the object’s interior. Figure 3 shows three examples where the heatmap closely surrounds the objects. This behavior is not unusual in self-supervised learning (SSL). Vision Transformers trained with DINO exhibit the same ability, and shape information remains robust across diverse transformations Caron et al. (2021). This observation suggests that shape-based cues play a fundamental role in the learned representations.

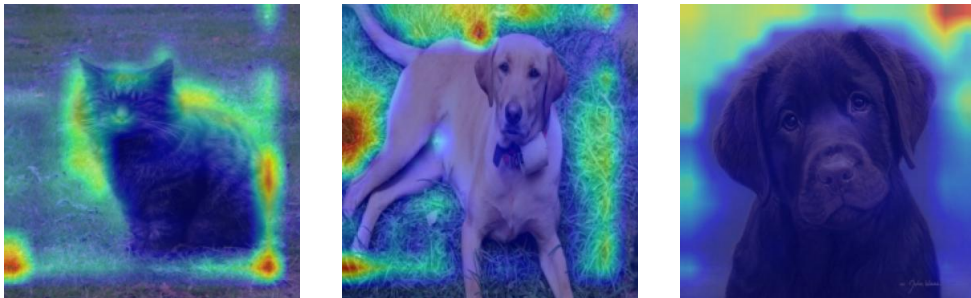


Figure 3: Target encoder identifies objects by their silhouette rather than their interior. Each patch is 14×14 pixels, forming a 16×16 occlusion grid.

2.2 Neuron Activation Distribution

Neuron activation distribution is an interpretability approach that examines how individual dimensions in a model’s embedding space vary across different inputs Bau et al. (2017). Rather than focusing on spatial patches or attention weights, this technique treats each embedding dimension as a potential feature detector. It investigates how its activation values are distributed for various classes or image sets. Researchers typically compute embeddings for a dataset and visualize the activation distributions to observe if certain dimensions respond strongly to specific classes or attributes. Sharp separation in one dimension across classes can indicate a specialized feature (e.g., “face detector”), while overlapping distributions suggest broader or more entangled representations. This method provides a concept-level view of the model’s learned features. It complements spatial methods like occlusion by revealing whether individual dimensions encode distinct concepts or share them more diffusely Zhou et al. (2018). By analyzing neuron activation distributions, we gain insight into the factorization of semantic information within high-dimensional embeddings, thereby clarifying the organization of a model’s representational capacity.

In the second milestone’s embedding-space experiment, we visualized 1,000 images spanning 10 classes (100 images per class) to examine high-level clustering in the learned representation. We now extend that approach by analyzing how each embedding dimension varies across these same images. Following Fisher (1970), we employ a one-way analysis of variance (ANOVA) to assess each dimension’s class-separating power. Specifically, for dimension j , we compute an F -statistic:

$$F_j = \frac{\sigma_B^2(j)}{\sigma_W^2(j)} \quad (3)$$

with $\sigma_B^2(j)$ being the between-class variance for dimension j defined as:

$$\sigma_B^2(j) = \frac{1}{K-1} \sum_{k=1}^K n_k (m_k^{(j)} - m^{(j)})^2 \quad (4)$$

and $\sigma_W^2(j)$ being the within-class variance for dimension j defined as:

$$\sigma_W^2(j) = \frac{1}{N-K} \sum_{k=1}^K \sum_{i \in C_k} (x_i^{(j)} - m_k^{(j)})^2. \quad (5)$$

Here, K is the number of classes, C_k is the set of sample indices in class k , n_k is the number of samples in class k , $x_i^{(j)}$ is the j -th dimension of sample i , $m_k^{(j)}$ is the mean of dimension j in class k , $m^{(j)}$ is the overall mean of dimension j , and N is the total number of samples. Higher F_j values indicate that dimension j exhibits greater inter-class variability relative to intra-class variability, suggesting it may encode distinctive features for certain classes. We rank all 768 dimensions by their respective F_j and focus on those with the largest values as the most discriminative.

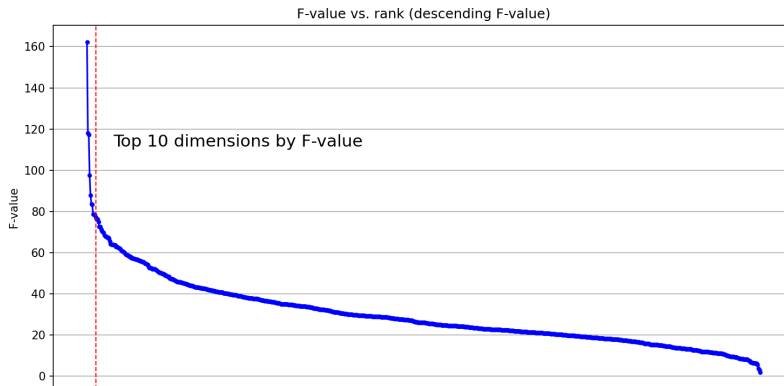


Figure 4: Distribution of F -values across embedding dimensions.

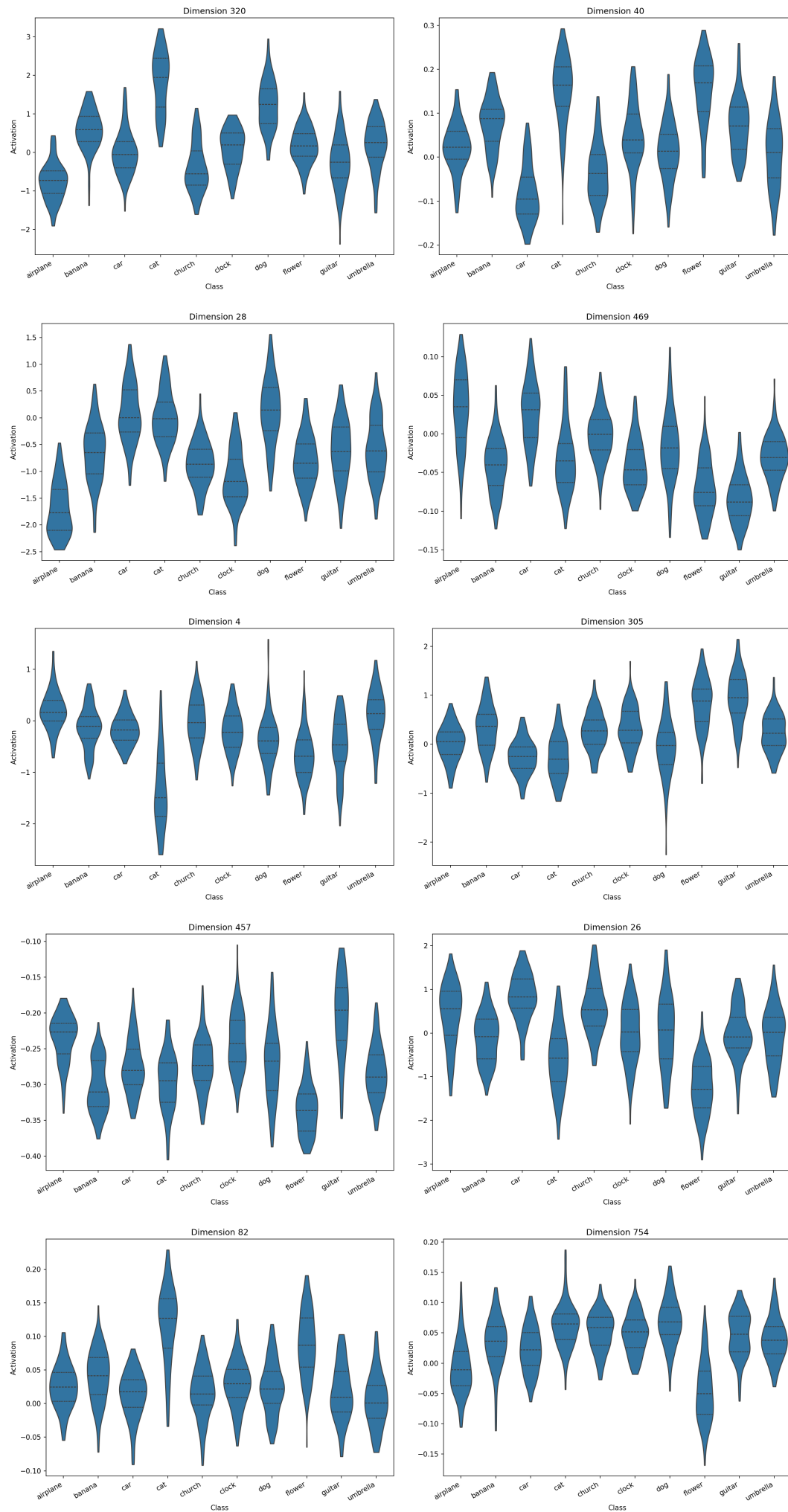


Figure 5: Neuron activation distributions across 10 classes.

Figure 4 shows the distribution of F -values for the target encoder’s 768 embedding dimensions (ViT-Base) in sorted order. The ten dimensions corresponding to the largest F_j stand out and promise the most class discriminative abilities. Hence, we focus on these dimensions for analyzing their discriminative characteristics. Figure 5 represents the neuron activation distributions for these dimensions across ten different classes. The order of the individual plots corresponds to their ranking in Figure 4 with $F_{320} = 162.36$ for embedding dimension 320. The violin plot for dimension 320 shows that the neuron activation distributions for classes cat and dog are shifted compared to the others. It indicates that this dimension captures a feature specific to those classes (e.g., animal fur). Similar patterns apply to other dimensions. Dimension 469 groups the classes of airplane and car together, indicating a common feature (e.g., the presence of wheels) not shared by the others. Dimension 40 activates more for classes cat and flower, indicating a possible shared feature. As we move to dimensions with smaller F -values (e.g., 754), the violin plots increasingly show mostly overlapping distributions, suggesting that individual dimensions alone provide less decisive class separation. This pattern indicates that I-JEPA’s learned features are distributed across multiple dimensions rather than cleanly encapsulated in any single one. Such distributed representations are common in self-supervised learning, where semantic attributes often emerge through the interplay of multiple embedding components rather than a single “feature neuron” Chen et al. (2020). Consequently, each dimension may capture only partial aspects of a concept, requiring a combination of dimensions to encode higher-level semantics fully.

2.3 Concept Activation Vector

Concept Activation Vectors (CAVs) provide a way to identify and test whether a model’s internal representations encode user-defined concepts Kim et al. (2018). Instead of analyzing individual neurons or spatial locations, CAVs directly capture a “direction” in the learned embedding space that corresponds to a high-level feature—such as “striped” or “round.” We derive a weight vector representing the concept’s axis by training a small binary classifier on embeddings containing or lacking a given concept. Any new image’s representation can then be projected onto this axis to measure how strongly the model associates that image with the concept, shedding light on how semantic information is organized within deep networks.

In the second milestone’s third experiment, we observed that I-JEPA’s target encoder showed poor invariance when images were converted to grayscale. This observation raised the question of whether the model can actually differentiate grayscale from color images at the level of its learned embeddings. To investigate, we first generated embeddings for 2,000 labeled images (1,000 color and 1,000 grayscale) using the target encoder. We drew our samples from 100 distinct classes (10 images per class for color and 10 images per class for grayscale). We then trained a logistic regression classifier on these embeddings by minimizing the following negative log-likelihood:

$$\min_{w,b} - \sum_{i=1}^N \left[y_i \log(\sigma(w^\top x_i + b)) + (1 - y_i) \log(1 - \sigma(w^\top x_i + b)) \right], \quad (6)$$

where (x_i, y_i) are the training samples, $y_i \in \{0, 1\}$ indicates the class, (w, b) are the parameters to be learned, and $\sigma(z)$ is the sigmoid function defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (7)$$

The learned weight vector w serves as the concept activation vector, and together with the bias term b , we evaluate unseen embeddings h by computing the logit score:

$$\text{score}(h) = w^\top h + b \quad (8)$$

In logistic regression, a decision boundary of zero implies that $\text{score}(x) > 0$ predicts one class (color), while $\text{score}(x) \leq 0$ predicts the other (grayscale). This threshold is valid because $\sigma(z)$ maps positive logits to probabilities above 0.5, distinguishing between color and grayscale based on the logit’s sign.

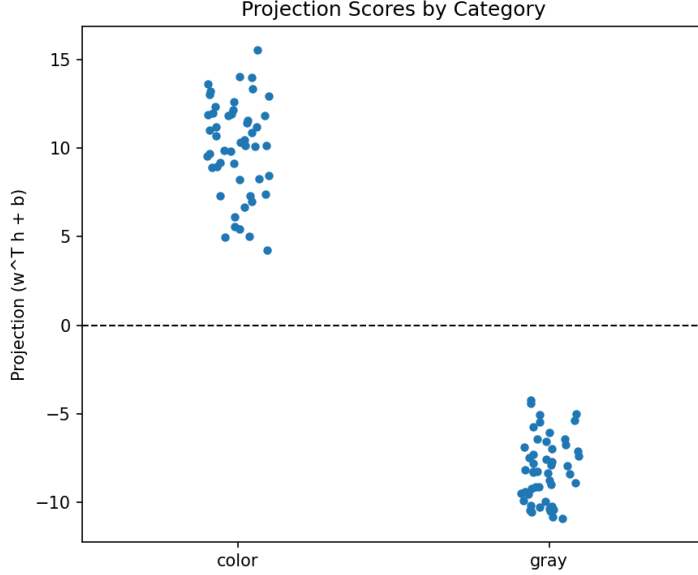


Figure 6: Projection scores on unseen images (50 color, 50 grayscale). The x-axis separates the two classes, and the y-axis shows the logit values ($w^T h + b$). Color images cluster at positive scores, while grayscale images cluster at negative scores, confirming a clear boundary in the embedding space and 100% classification accuracy.

We evaluate I-JEPA’s ability to distinguish between color and grayscale embeddings by testing the learned concept vector w on 100 previously unseen images, evenly split between color and grayscale. Figure 6 illustrates the resulting projections ($w^T h + b$) for each image, with the x-axis indicating the two classes and the y-axis showing the logit scores. All color images produced positive scores, and all grayscale images were negative, yielding a 100% classification accuracy. This outcome suggests that I-JEPA’s embeddings capture robust color information despite our initial concerns prompted by the model’s weaker invariance to grayscale augmentations. The apparent discrepancy arises because transforming an image to grayscale can alter specific embedding dimensions more strongly than other augmentations, effectively establishing a distinct boundary in the learned representation Zhang et al. (2016). Consequently, while I-JEPA struggles to preserve embedding similarity under grayscale transformations, it still learns sufficient color-related features to separate them with robust reliability. We conclude that the concept “grayscale” is distinctly present in the learned representation space.

2.4 Multi-class Concept Activation

In experiment 2.3, we studied a binary concept (grayscale vs. color) to see if I-JEPA’s target encoder could separate those two classes. We now extend that idea to multiple classes, testing whether I-JEPA’s embeddings distinguish different rotation angles of the same images. This experiment focuses on the model’s rotation invariance, especially in light of earlier results (Milestone 2, third experiment), where moderate rotations preserved embedding similarity.

The authors emphasize that their method does not rely on extensive augmentations for training, unlike some other SSL approaches. Nevertheless, they still observe competitive results at fewer epochs Assran et al. (2023). If rotation is truly invariant within the learned representation, we would expect only minor changes in the embeddings when an image rotates 15°, 30°, or 45°. However, if the embeddings capture orientation cues, we can train a simple classifier to distinguish these angles accurately, indicating that I-JEPA learns rotation-related information in its representation.

We selected 1,000 original images (10 images per class from 100 different ImageNet1k classes). We generated three augmented copies for each image at rotation angles of 15°, 30°, and 45°. We passed every image through the frozen I-JEPA target encoder to obtain embeddings. Once we had 4,000 labeled embeddings—organized

into four rotation classes (0° , 15° , 30° , 45°)—we fit a softmax logistic regression to learn four sets of parameters $\{(w_0, b_0), (w_{15}, b_{15}), (w_{30}, b_{30}), (w_{45}, b_{45})\}$. Similar to the binary case but extended to multiple classes, we minimize the cross-entropy loss:

$$\min_{\{w_j, b_j\}} - \sum_{i=1}^N \sum_{j=1}^4 y_i^{(j)} \log \left(\sigma_j(w_1^\top x_i + b_1, \dots, w_4^\top x_i + b_4) \right), \quad (9)$$

where $y_i^{(j)}$ is a one-hot label for image i , and σ_j represents the softmax for class j :

$$\sigma_j(z_1, z_2, z_3, z_4) = \frac{\exp(z_j)}{\sum_{k=1}^4 \exp(z_k)} \quad (10)$$

Here, $z_j = w_j^\top x_i + b_j$ denotes the logit for class j . We interpret a higher z_j as a greater likelihood for rotation class j . During inference on unseen image-embeddings h , we compute four logit scores:

$$\text{score}_j(h) = w_j^\top h + b_j, \quad j \in \{0, 15, 30, 45\} \quad (11)$$

We then apply the softmax function and assign the image to whichever class yields the highest score (i.e., highest softmax probability). For example, if $\text{score}_{30}(h)$ exceeds all others, the classifier predicts a 30° rotation.

This experiment determines if the target encoder’s representations remain invariant to mild and moderate rotations. If the embeddings do not significantly change across angles, the softmax logistic regression should not classify these rotations accurately. On the other hand, if the model encodes orientation cues, the classifier will quickly identify each angle. Thus, this multi-class approach provides direct evidence of whether I-JEPA implicitly learns rotation information—even though it was never explicitly trained on rotation augmentations Assran et al. (2023).

Figure 7 shows the confusion matrix for our rotation classifier, revealing an overall accuracy of 83.5%. This outcome indicates that although I-JEPA was never explicitly trained to recognize rotated inputs, its target encoder still captures orientation cues well enough for a simple linear model to discriminate four distinct angles. In other words, rotation is not strictly invariant in the learned representation: small and moderate rotations leave discernible imprints in the embedding space. At the same time, this result reflects a broader capability of I-JEPA to encode features related to image geometry despite lacking any rotation-specific augmentation during its self-supervised pre-training.

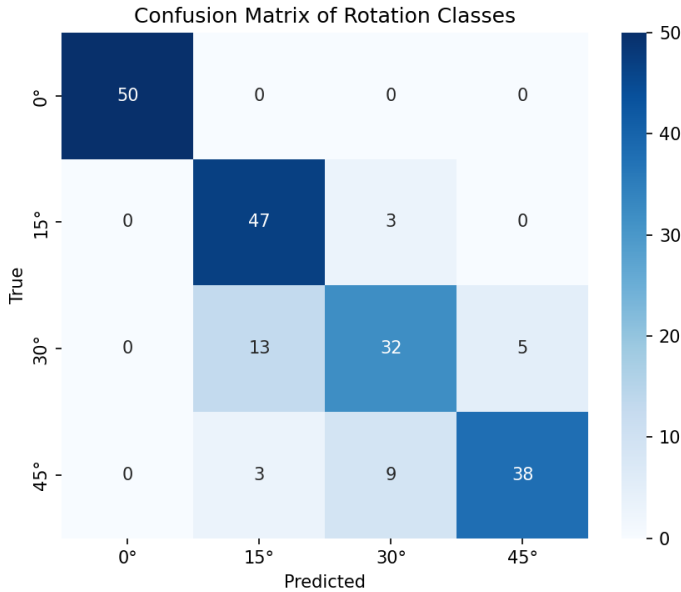


Figure 7: Confusion matrix for the multi-class rotation experiment. Each row represents the true rotation angle, while each column indicates the predicted class.

References

- M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023.
- D. Bau, B. Zhou, A. Khosla, A. Oliva, and A. Torralba. Network dissection: Quantifying interpretability of deep visual representations. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6541–6549, 2017.
- M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.
- T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020.
- A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- R. A. Fisher. Statistical methods for research workers. In *Breakthroughs in statistics: Methodology and distribution*, pages 66–70. Springer, 1970.
- R. C. Fong and A. Vedaldi. Interpretable explanations of black boxes by meaningful perturbation. In *Proceedings of the IEEE international conference on computer vision*, pages 3429–3437, 2017.
- B. Kim, M. Wattenberg, J. Gilmer, C. Cai, J. Wexler, F. Viegas, et al. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (tcav). In *International conference on machine learning*, pages 2668–2677. PMLR, 2018.
- K. Xu. Show, attend and tell: Neural image caption generation with visual attention. *arXiv preprint arXiv:1502.03044*, 2015.
- M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional networks. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part I 13*, pages 818–833. Springer, 2014.
- R. Zhang, P. Isola, and A. A. Efros. Colorful image colorization. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part III 14*, pages 649–666. Springer, 2016.
- B. Zhou, Y. Sun, D. Bau, and A. Torralba. Revisiting the importance of individual units in cnns via ablation. *arXiv preprint arXiv:1806.02891*, 2018.

3 Appendix



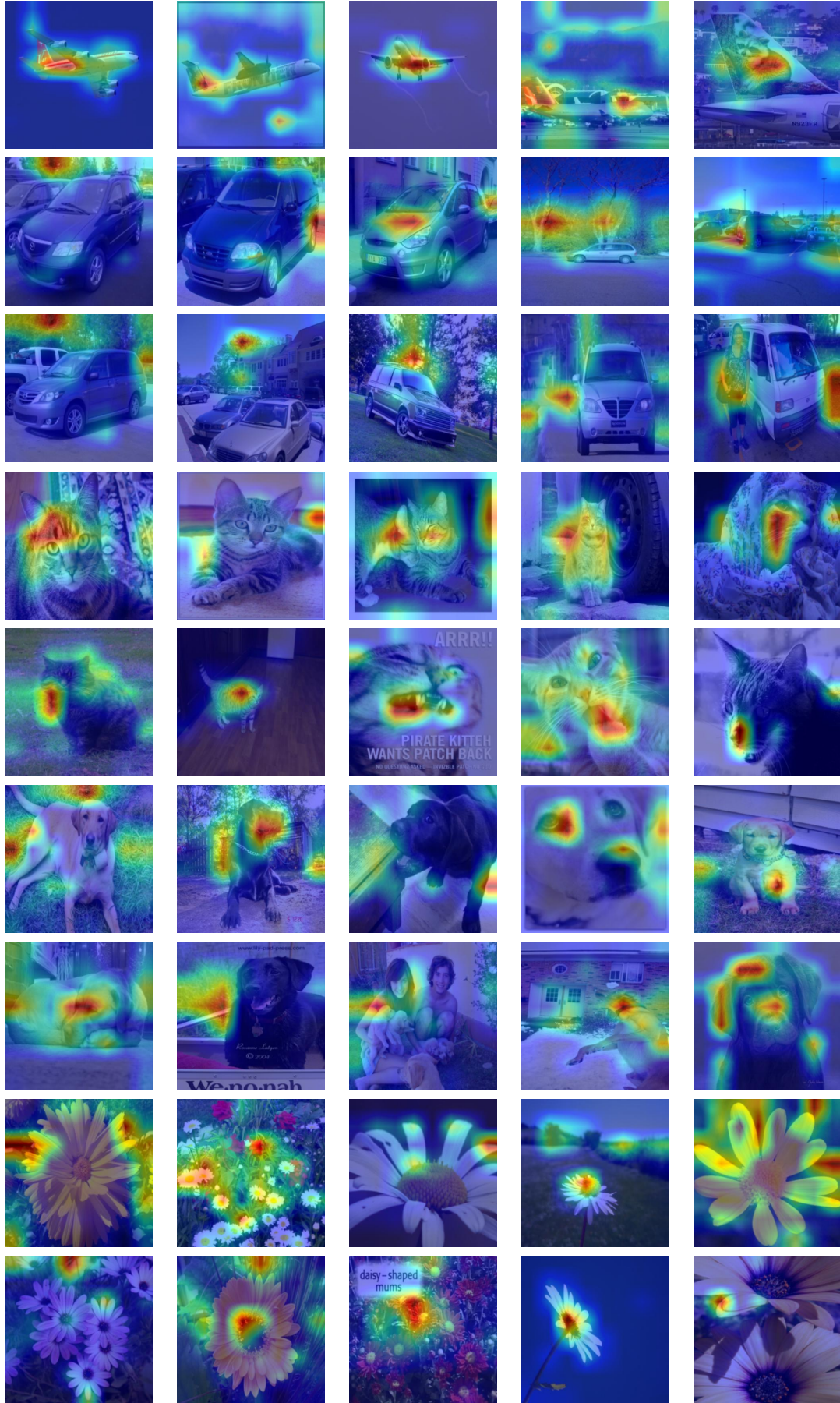


Figure 8: Heatmaps of all images from Milestone 2, Experiment 1. Each patch measures 28×28 pixels, forming an 8×8 occlusion grid.